Day 81: The Whole Transformer — One Forward Pass on a Toy Copy Task

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 5 | Complete study guide

Watch: Illustrated Guide to Transformers Neural Network: A step by step explanation (The AI Hacker) | Attention is all you need (Transformer): Model explanation, Inference and Training (Umar Jamil)

Script: `day81_transformer_forward.py` · every number in this guide is copied by program from `day81_run_output.txt`, the real output of that script.

Days 77–80 built the Transformer's parts. Today they are joined into the complete encoder-decoder model and run forward, untrained, on Day 69's digit-copying task. Before any weight is learned the assembled model already keeps its promises: padded source positions change nothing (0.00e+00), a later target token never moves an earlier prediction, and writing one token at a time gives exactly the scores of one teacher-forced pass (2.66e-15). One thing it does not do yet is start from even bets: its untrained loss is about 2.8, not log 10 = 2.303, because of how the output layer starts.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

The **Transformer** (Vaswani et al., 2017) is an **encoder** that reads the whole source with self-attention, and a **decoder** that writes the output one token at a time: its masked self-attention looks at what it has written so far, its cross-attention reads the encoder's output, and a final linear layer and softmax give the probability of every possible next token. **Teacher forcing** gives the decoder the correct previous tokens during training, so every position can be scored in one pass.

REAL-WORLD SCENARIO

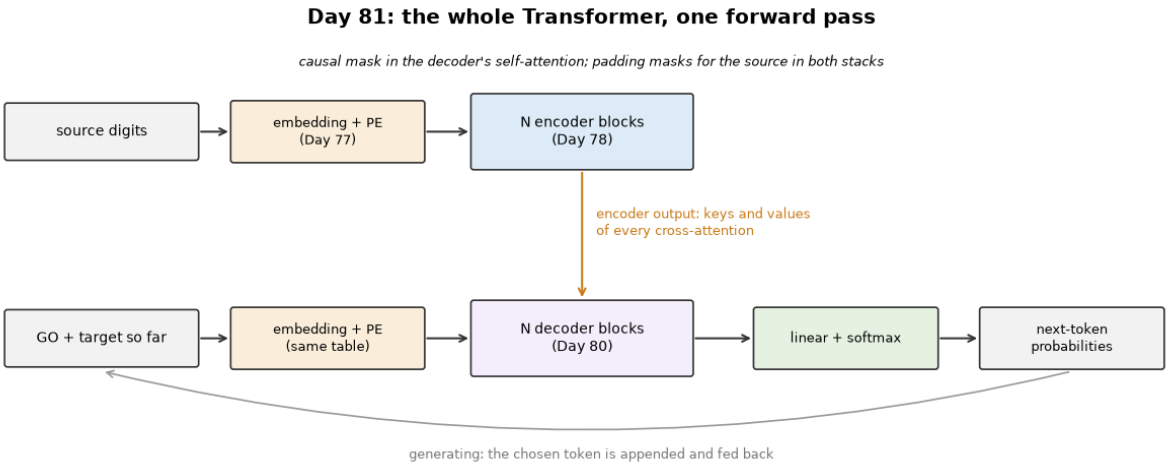
A translator who first reads the whole sentence (the encoder), then writes the translation word by word, rereading their own draft (masked self-attention) and glancing back at the original (cross-attention) before each new word.

MATHEMATICS

$$enc = Encoder(\ E[source] + PE\ ),\quad dec = Decoder(\ E[GO, y_1, ..., y_t] + PE, enc\ )$$

$$p(y_{t+1} \mid y_{\le t}, source) = softmax(\ dec_t\, W_{out} + b_{out}\ )$$

SYSTEM DIAGRAM



The whole Transformer, with the week's parts named.

Today's part for Day 83

`day81_exercise.py` joins the earlier exercise functions into `transformer_forward`, with a new `decoder_block`: Week 12's fifth part.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. One example through the whole model

DEFINITION — start here, no prior knowledge assumed

One REVERSE example of 5 digits through an untrained model with one encoder block and one decoder block, with the shape of the numbers at every stage.

REAL-WORLD SCENARIO

Following one parcel through every station of a sorting office.

SPECIFICATION

Model	32 numbers per token, 4 heads, FFN 64, one block per side; one embedding table for source and target.
Data	Day 69's task: digits 1–8, EOS = 0, GO = 9; the decoder input is GO plus the target, shifted.

Why this test

Shapes are where an assembled model first goes wrong: every stage must hand the next what it expects.

IMPLEMENTATION

```
class Transformer:
    def __init__(self, n_enc=1, n_dec=1, seed=0, embed_scale=1.0, out_scale=1.0):
        rng = np.random.RandomState(seed)
        self.P = dict(E=rng.randn(V, D) * embed_scale)
        self.enc = [EncoderBlock(rng) for _ in range(n_enc)]
        self.dec = [DecoderBlock(rng, causal=True, cross=True) for _ in range(n_dec)]
        self.P.update(Wout=rng.randn(D, V) / np.sqrt(D) * out_scale, bout=np.zeros(V))
        self.PE = sinusoidal_encoding(MAX_LEN, D)
    def params(self):
        out = dict(self.P)
        for i, b in enumerate(self.enc):
            out.update({f"e{i}.{k}": v for k, v in b.P.items()})
        for i, b in enumerate(self.dec):
            out.update({f"d{i}.{k}": v for k, v in b.P.items()})
        return out
    def encode(self, src, smask):
        x = self.P["E"][src] + self.PE[:src.shape[1]]
        km = head_mask(smask, HEADS)
        for b in self.enc:
            x = b.forward(x, 0.0, km)
        return x
    def decode(self, tgt_in, tmask, enc, smask):
        y = self.P["E"][tgt_in] + self.PE[:tgt_in.shape[1]]
        for b in self.dec:
            y = b.forward(y, 0.0, tmask, enc, smask)
        return y
    def forward(self, src, smask, tgt_in, tmask):
        self.enc_out = self.encode(src, smask)
        self.dec_out = self.decode(tgt_in, tmask, self.enc_out, smask)
        logits = self.dec_out.reshape(-1, D) @ self.P["Wout"] + self.P["bout"]
        logp = logits - logits.max(axis=1, keepdims=True)
        return logp - np.log(np.exp(logp).sum(axis=1, keepdims=True))
```

ACTUAL OUTPUT

```
source ids [6, 4, 5, 1, 8]   decoder input [9, 8, 1, 5, 4, 6]   target [8, 1, 5, 4, 6, 0]
source embeddings + positions   (1, 5, 32)
encoder output                  (1, 5, 32)
decoder embeddings + positions  (1, 6, 32)
decoder output                  (1, 6, 32)
cross-attention weights, per head (decoder positions x source positions): (6, 5)
scores for the next token       (6, 10)  (6 positions x 10 tokens)
untrained prediction at each position: [9, 9, 9, 9, 7, 3]
Actual output from this lesson's run.
```

What it means

The encoder keeps one vector per source digit; the decoder keeps one per position it writes; cross-attention weighs 6 decoder positions against 5 source positions; and every decoder position ends with 10 scores, one per token. Untrained, the model mostly predicts GO (token 9), a token that never appears in a correct output.

WHY CHOOSE THIS?

Every stage's shape checked on one real example.

WHY NOT THIS? (TRADEOFFS)

Says nothing yet about whether the model can learn.

2. Where the weights are

DEFINITION — start here, no prior knowledge assumed

The weight counts by part, for one and for two blocks on each side.

REAL-WORLD SCENARIO

The budget of a building: how much goes to each floor.

ACTUAL OUTPUT

```
1 + 1 blocks: embeddings 320, encoder 8416, decoder 12576, output layer 330; total 21642
  one decoder block: self-attention 4096, cross-attention 4096, FFN 4192, LayerNorms 192
2 + 2 blocks: embeddings 320, encoder 16832, decoder 25152, output layer 330; total 42634
Actual output from this lesson's run.
```

What it means

With one block per side the model has 21642 weights. A decoder block is about one and a half encoder blocks (12576 against 8416): it has a second attention, the cross-attention (4096 weights). The embeddings and the output layer are tiny (320 and 330) because the vocabulary has only 10 tokens.

WHY CHOOSE THIS?

Shows where model size really comes from.

WHY NOT THIS? (TRADEOFFS)

A tiny vocabulary hides how large the embeddings become with real text.

3. The masks, end to end

DEFINITION — start here, no prior knowledge assumed

Three changes to the input of the untrained model: digits placed under source padding, a different decoder input at position 5, and a different real source digit.

REAL-WORLD SCENARIO

Changing what is written on an unused page, on tomorrow's page, and on today's page of a diary.

Why this test

Each block's masks were checked on its own (Days 78, 80). Here they are checked through the whole stack.

ACTUAL OUTPUT

```
(a) change the PADDED source positions:          every score changes by at most 0.00e+00
(b) change decoder input 5: scores at positions 0-4 change by 0.00e+00, at 5-8 by up to 2.59e+00
(c) change a REAL source digit: scores change at every decoder position, smallest change 1.92e-01
Actual output from this lesson's run.
```

What it means

(a) Padding is invisible through both stacks (0.00e+00). (b) A later decoder input never moves an earlier score; it moves later ones. (c) A real source digit reaches every decoder position, the smallest change being 1.92e-01: cross-attention makes the whole source available to every output.

WHY CHOOSE THIS?

The guarantees hold for the assembled model.

WHY NOT THIS? (TRADEOFFS)

Untrained weights; the guarantees do not depend on them.

4. Before any training: the loss against log V

DEFINITION — start here, no prior knowledge assumed

An untrained model should be unsure: equal probability for each of the 10 tokens gives a loss of log 10. The untrained loss for three embedding spreads, and for an output layer that starts 10 times smaller.

REAL-WORLD SCENARIO

A new weather forecaster should say 'fifty-fifty', not 'certainly rain', before they have seen any weather.

MATHEMATICS

even bets: loss = –log(1/V) = log V = 2.303 for V = 10

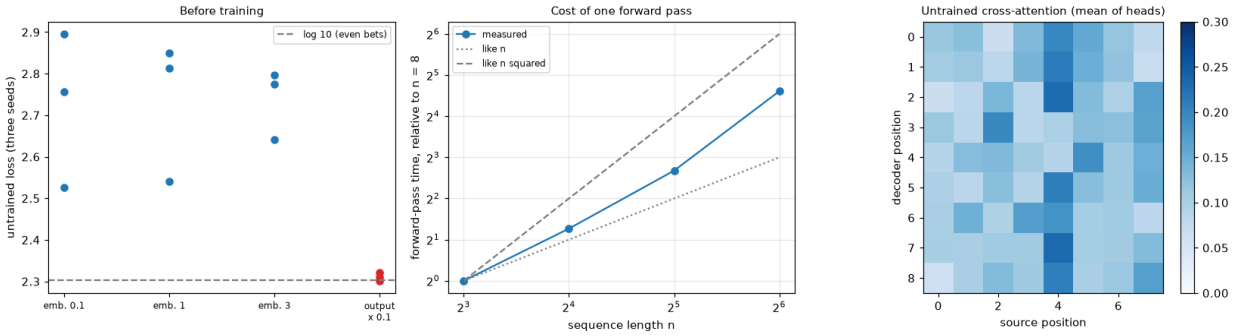
logits = LN(dec) W_{out}: LayerNorm fixes the spread of LN(dec) at about 1, so W_{out} alone sets how spread the logits are

Why this test

A model that starts confident and wrong begins training with a needless penalty.

ACTUAL OUTPUT

```
model                          seed 0   seed 1   seed 2   largest probability (mean)
embedding spread 0.1            2.895    2.527    2.756                        0.305
embedding spread 1              2.849    2.541    2.813                        0.299
embedding spread 3              2.796    2.641    2.775                        0.308
output layer x 0.1              2.322    2.302    2.310                        0.115
Actual output from this lesson's run.
```



Left: the untrained loss (three seeds) against log 10. Middle: forward-pass time against sequence length (Section 6). Right: the untrained model's cross-attention on one REVERSE example, mean of the heads.

What it means

The embedding spread makes no difference (2.895 to 2.527 at 0.1, 2.796 to 2.641 at 3): the decoder ends in a LayerNorm, which gives its output the same spread whatever came in. **The output layer does:** starting it 10 times smaller brings the loss to 2.322, 2.302 and 2.310, and the largest probability from about 0.30 down to 0.115. Day 83 will start from the smaller output layer.

WHY CHOOSE THIS?

A cheap check that the model starts unbiased.

WHY NOT THIS? (TRADEOFFS)

The fix is a choice of initial scale, not a law.

5. Greedy decoding, and why one pass is enough in training

DEFINITION — start here, no prior knowledge assumed

Greedy decoding writes one token at a time, each time choosing the most probable token and feeding it back. The tokens it wrote are then given to the decoder all at once, in one pass.

REAL-WORLD SCENARIO

Writing a sentence word by word, then rereading the finished sentence: each word's context is the same either way.

Why this test

Training scores all positions in one pass; using the model writes them one by one. The two must agree.

IMPLEMENTATION

```
def generate(self, src, n_steps):
    m = len(src); smask = np.ones(src.shape, bool)
    enc = self.encode(src, smask)
    tokens = np.full((m, 1), GO)
    steps = []
    for _ in range(n_steps):
        y = self.decode(tokens, np.ones(tokens.shape, bool), enc, smask)
        logits = y[:, -1] @ self.P["Wout"] + self.P["bout"]
        lp = logits - logits.max(axis=1, keepdims=True); lp -= np.log(np.exp(lp).sum(axis=1,
            keepdims=True))
        steps.append(lp)
        tokens = np.concatenate([tokens, lp.argmax(axis=1)[:, None]], axis=1)
    return tokens[:, 1:], np.stack(steps, axis=1)
```

ACTUAL OUTPUT

```
largest difference between the two: 2.66e-15; same tokens chosen: True
untrained model: exact match 0.000, token accuracy 0.040; its most written token is 9 (0.573 of
    all tokens)
Actual output from this lesson's run.
```

What it means

They agree to rounding (2.66e-15) and choose the same tokens: the causal mask makes training's single pass a faithful copy of decoding. Untrained, the model gets 0.040 of the tokens right and writes GO 0.573 of the time.

WHY CHOOSE THIS?

One pass per sentence in training is exactly right.

WHY NOT THIS? (TRADEOFFS)

This decoder recomputes the whole prefix at every step; real systems cache the keys and values.

6. What a forward pass costs as sequences grow

DEFINITION — start here, no prior knowledge assumed

The time of one forward pass at lengths 8, 16, 32 and 64.

REAL-WORLD SCENARIO

Doubling the guests at a dinner where everyone must speak to everyone: the conversations grow faster than the guests.

MATHEMATICS

$$\text{projections and FFN: work} \propto n; \quad \text{attention scores } Q\,K^T\text{: work} \propto n^2$$

ACTUAL OUTPUT

```
   n  ms per pass  vs n = 8
   8         1.84      1.00
  16         4.42      2.41
  32        11.76      6.40
  64        44.85     24.41
(doubling n would double the cost if everything grew like n, and quadruple it if everything grew
    like n squared)
Actual output from this lesson's run.
```

What it means

Eight times the length cost 24.41 times as much here: more than the 8 times of work that grows like n, less than the 64 times of work that grows like n squared. At these small lengths the parts that grow like n still take a large share. Only ratios within one run are meaningful (this machine's speed varies between runs).

WHY CHOOSE THIS?

Attention is cheap at short lengths.

WHY NOT THIS? (TRADEOFFS)

Its n-squared part dominates as sequences grow long.

Interview preparation — technical questions

Q: Name the stages of a Transformer forward pass.

A: Embedding + PE, encoder blocks, embedding + PE of the target so far, decoder blocks (masked self-attention, cross-attention, FFN), linear, softmax.

Q: What is teacher forcing?

A: Giving the decoder the correct previous tokens in training, so every position is scored in one pass.

Q: Why is one teacher-forced pass the same as decoding step by step?

A: The causal mask: each position sees only earlier ones (2.66e-15 here).

Q: What should the untrained loss be?

A: About log V; here 2.303, reached only with a small output layer.

Q: How does the cost grow with length?

A: Partly like n, partly like n squared (attention); 8x the length cost 24.41x here.

Interview preparation — theoretical questions

Q: Why can the source and target share an embedding table here?

A: They are the same digits; with different languages they would differ.

Q: Why did the embedding spread not change the untrained loss?

A: The decoder ends in LayerNorm, which fixes the spread of what reaches the output layer.

Q: Why check masks again after checking each block?

A: Assembly can break them: a wrong mask passed between stacks would only show end to end.

Q: What does greedy decoding give up?

A: It never revisits a choice; beam search keeps several candidates.

Key takeaway and what's next

> **KEY TAKEAWAY**
> Joined together, the week's parts form a complete Transformer whose guarantees hold end to end before training: padding is invisible, the future is invisible, the source reaches every output, and one teacher-forced pass equals decoding token by token. The one adjustment it needs is a smaller output layer, so that it starts from even bets.

- 21642 weights with one block per side; a decoder block 12576, an encoder block 8416.
- Source padding: 0.00e+00; a later target token: 0.00e+00 on earlier scores.
- Untrained loss about 2.5-2.9 whatever the embedding spread; 2.302-2.322 with the output layer x 0.1.
- Decoding token by token equals one pass: 2.66e-15.

Suggested watch order, if starting from zero

- 1. Illustrated Guide to Transformers Neural Network: A step by step explanation (The AI Hacker) — the whole model, picture by picture.
- 2. Attention is all you need (Transformer): Model explanation, Inference and Training (Umar Jamil) — the same with the maths, inference and training.

Day 82 preview

- **The backward pass through the whole model.** Each block's backward pass exists; Day 82 chains them, from the loss back to the shared embedding table, and checks every gradient numerically.
- Today's exercise script, `day81_exercise.py`, is the forward pass built from the week's exercise parts: the fifth part of Day 83's Transformer.